Accepted for publication at:18th IEEE International Conference on Machine Learning and Applications - ICMLA 2019, Dec 16-19, Florida, USA

Hydropower Optimization Using Split-window, Meta-Heuristic and Genetic Algorithms

Jivitesh Sharma*, Bernt Viggo Matheussen†, Sondre Glimsdal‡, and Ole-Christoffer Granmo§
* † ‡ § Centre for Artificial Intelligence Research
* ‡ §*University of Agder, Norway*
†*Agder Energi AS, Norway*
*jivitesh.sharma@uia.no, †bernt.viggo.matheussen@ae.no, ‡sondre.glimsdal@uia.no §ole.granmo@uia.no

***Abstract*—In this paper, we try to find the most efficient optimization algorithm that can be used to resolve the hydropower optimization problem. We propose a novel optimization technique called the Split-window method. The method is relatively simple and reduces the complexity of the optimization problem by splitting the planning horizon (and datasets) into equal windows and assigning the same values to policies(actions) within each part. After splitting, a meta-heuristic technique is used to optimize the actions, and the dataset is split again until a split contains only one instance (timestep). The unique values to be optimized during each iteration is equal to the number of splits which makes it very fast and requires fewer computations. We also propose a novel initialization method based on ranking of price and assigning a higher value of production and hatch release for higher prices. We apply this initialization technique to most of the algorithms used in this paper. We compare the split-window technique with meta-heuristic methods such as hill climbing, simulated annealing, line search, and genetic algorithms by running simulations on the data collected from a real-world hydropower river system in southern Norway. In total, we benchmark the performance of seven different optimization algorithms for a large number of hydrological and price scenarios. The results show that the Split-window method is able to beat other techniques in terms of performance score, speed of convergence and core algorithmic complexity by a considerable margin.**

***Index Terms*—Hydropower, Hydrology, Split-window, Genetic Algorithm, Simulated Annealing, Hill Climbing, Optimization, Meta-heuristics.**

I. Introduction

Hydropower reservoir operations have been one of the major industrial applications of optimization methods. Both researchers and practitioners have worked on this problem for many decades, but the success has been elusive [23]. The issue with reservoir optimization is extraordinarily complex due to numerous varying parameters such as: uncertainties in weather patterns and daily shifts in power pricing. Additionally, there are environmental and reservoir constraints, flooding risks associated with societal costs, and more, that must be considered during the short- and long-term planning horizons.

In Norway roughly 95 % of the domestic power consumption is derived from hydropower. Norway is part of the Nordic electricity market [17] and many of the riversystems consists of multi-reservoir powerplants with complicated hydrologic properties and hydraulic structures. As a consequence of this production planners have a need for decision support tools that are robust, efficient and can find realistic optima for given price and inflow situations. The Scandinavian industry have applied various mathematical programming techniques to resolve these issues [15], but there is still a great need for improvement. The motivation behind this research is to continue the quest of identifying improved methods that can be used for better reservoir operations and scheduling in the Nordic region.

The rest of this paper has been organized into the following sections: Section 2 enlists and briefly mentions the past research works relevant to this paper. Section 3 details the study area and available data used in this research. Section 4 defines the deterministic optimization problem that mus be resolved for a given price and inflow scenario. Section 5 explains all the algorithms proposed and tested for optimization of hydropower generation. Section 6 shows the conducted experiments and displays results obtained from exhaustive testing of the algorithms used. Finally, section 7 concludes our paper with a short summary of our work and future prospects.

This research article is a continuation of earlier work by three of the authors [4].

II. Related Work

The literature within the field of reservoir management and operations is vast and comprehensive. Several review articles are available, and the reader is referred to [1], [7], [21], [23] for in-depth insight into the current status of the field.

Due to a large number of methods, attempts have been made to categorize them into groups that have similar features. The grouping may be dependent on several criteria such as if the technique is deterministic or stochastic if it uses mathematical programming, heuristic methods if it's simulation-based and so on. In this research paper, we will try to mention the most important and relevant literature within all these categories, but due to page constraints, it cannot be comprehensive.

Amongst many techniques available today, many linear and dynamic programming based techniques seem to have gained momentum compared to other methods. Some research works employing these methods are linear programming (LP) [15], [24], non-linear programming (NLP) [19], [22] and dynamic

programming (DP) [3], [20]. Unfortunately, these traditional methods have severe drawbacks. Most notably are the well-known problems referred to as the “curse of dimensionality and modeling“ [6].

Linear programming methods, for instance, face difficulties with uncertain information, requiring extensive Monte Carlo simulation. Furthermore, the hydraulic head may not be captured by linear models since it is intrinsically non-linear in its behavior. Moreover, computational complexity multiplies with both the number of system components and the extent of the time horizon. Dynamic programming, on the other hand, handles input with uncertainty. They scale linearly in the time dimension and capture non-linear and uncertain relationships. Moreover, computational complexity grows exponentially with the number of system components, making this class of schemes incapable of modeling larger systems in full detail. Temporal resolutions are often coarse (weekly, monthly), which results in an underestimation of flooding in smaller and steep catchments, resulting in sub-optimal solutions. Finally, in an operational setting, computation time is crucial for decision making since the production planners operate in time-constrained markets.

In an effort to mitigate some of the challenges related to the classical techniques, a huge range of different meta-heuristic methods have been applied to the hydropower optimization problem. These methods typically utilize different concepts for exploring and exploiting the search space inspired by various biologically or ”nature” based phenomenons. Often they are used in combination with simulation models of the river system under consideration.

A recent paper [8] applies the so-called Firefly algorithm (FA) to the hydropower optimization problem. The algorithm is motivated by the grouping behaviour of fireflies, and has strong similarities to the particle swarm method [6]. The authors show that the Firefly algorithm is superior compared to genetic algorithms (GA), when it comes to resolving the reservoir management problem.

Another heuristic inspired method is the Big-Bang Big-Crunch (BB-BC) algorithm [14] which is a population-based algorithm that relies on one of the theories of the evolution of universe; namely, the Big Bang and Big Crunch theory. The method showed comparable results to genetic algorithms and particle swarm method for a study system in Iran. Another meta-heuristic method is the invasive weed optimization (IWO) algorithm [12]. IWO is a novel evolutionary algorithm inspired from colonizing weeds. Other methods include search algorithms such as gradient ascent/descent and simulated annealing. For more on these, please see [11], [14], [18], [23] and citations therein.

Machine learning (ML) has further been used in water resources management. Labadie [23] refers to work starting in the 1970s shaping ideas on how to incorporate various ML concepts into the reservoir management problem. After that, a wide range of methodologies have appeared in the literature trying to improve either deterministic or stochastic methods. For example, DeRigo tried to use neural networks to estimate the Bellman function in stochastic dynamic programming [5]. Use of the Q-Learning method in reinforcement learning was demonstrated by [10] on a two-reservoir river system. They claim that the method outperformed more classical techniques. Such an approach was further investigated and modified by [2] who used a tree-based reinforcement learning algorithm trying to identify optimal water reservoir operation for the Lake Como river system in Italy. The results showed improved performance compared to traditional (stochastic) DP methods. More recently Dariane used neural networks and reinforcement learning in combination with a meta-heuristic method (particle swarm) to optimize a river-system in Iran [6]. Lately, Sangiorgo used a Neural Network-based implicit stochastic optimization (ISO) technique to optimize the Nile multi-reservoir system [13]. The results demonstrated among other things that Neural Networks can reduce computational demand, facilitating real-time operation.

From the literature cited above we can see that different ML techniques have been applied to reservoir management problems around the world. Nevertheless, the application of these techniques to river systems located in the Nordic region is absent. The reason for this is not clear, but in the Nordics all power stations are connected to a competitive electricity market, thus both price and inflow must be accounted for in the optimization procedure. It should be noted that decision support tools based on the well-known LP and DP methods have been used by the industry for more than a decade for this purpose [15].

III. Study Area and Available Data

In this research the hydropower system of Kvinesdal, located in southern-Norway, is used as a case study. A location map, and schematics of the system is shown in Fig. 1. The system

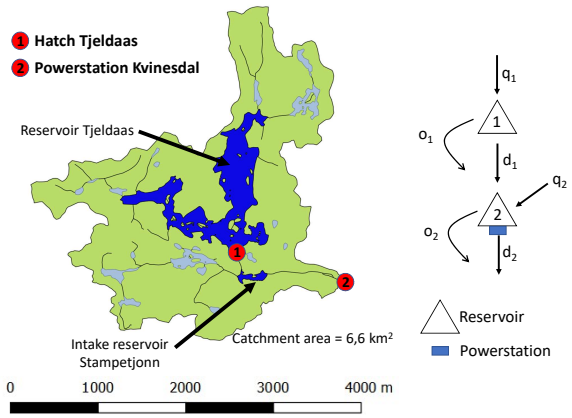


Fig. 1. Schematic of the Kvinesdal hydropower river system (southern-Norway).

consists of two reservoirs and one hydropower station. The uppermost reservoir, named Tjeldaasvatn, is laying at an average elevation of 312 meters above sea level (m.a.s.l). The reservoir

is connected to the downstream reservoir Stampetjonn through an open channel. The water release out of Tjeldaasvatn is controlled by a hatch that can release up to 1 m^3/s of water. The lower most reservoir Stampetjonn serves as an intake reservoir to the Kvinesdal power plant. It is located at an average elevation of 302 m.a.s.l. Both the upper and lower reservoir are constructed with overflow safety spillways that transports water into the old river bed during high water levels (above upper regulated water heights). During operations, water from the intake reservoir is transported through a $\sim 1km$ circular pipe down to the power station at 38 m.a.s.l. Kvinesdal power plant has an installed maximum power capacity of 1.4 MW at a water usage of around 0.69 m^3/s. This is in Norwegian scale considered to be a small station. The powerplant is owned and operated by Agder Energi AS - a Norwegian provider of renewable energy. The plant is connected to the Nordic electricity grid and is part of the Nordic physical electricity market.

In this research, time series of price and inflow to the Kvinesdal system was provided from the historical archive of Agder Energi. The historical records covered the time period from January 1, 2007 until August 31, 2017 and had a temporal resolution of 24 hours. It should be noted that the inflow data used in this work represents the inflow to the whole river system (catchment area 6.6 km^2). In the hydraulic calculations of the river system local inflow to the system must be provided for both the upper and the lower reservoirs. This was resolved by splitting the inflow data in two time series scaled after the contributing area to the reservoirs. An effect of this is that both the upper and lower reservoirs receives local natural inflow at the exact time. It was chosen to neglect this effect.

IV. The Optimization problem

In this work a scenario is defined as a time-series of inflow and price with a specified temporal resolution and planning horizon (e.g. 24 hours and 180 steps/days). In addition the starting reservoir levels, and expected rest price after the planning horizon, is also part of the scenario. The optimization problem is therefore to find the most profitable combination of water release decisions (actions, e.g. hatch release and production level) for each of the time-steps in the planning horizon. Equation 1 to 6 defines the deterministic optimization problem that must be resolved for a given scenario. For each hydraulic node (reservoir or powerstation) in the system (n), profit (π) is calculated as the income ($e_t * w_t$), then subtracting the operational costs (c). We also model the effect of an operation policy (action) that violates operational constraints such as minimum reservoir levels, or start-stop costs of machinery. In practice this would also include minimum flow requirements and other environmental constraints, but this is not included in this work. The value of the remaining water (I) in the reservoirs, at the end of the planning horizon is estimated assuming a rest price (e_{T+1}), and an average reservoir height. The value is added to the total profit for each of the hydraulic nodes (power station or reservoir) in the system.

$$Max(F), \qquad F = \sum_{n=1}^{N} \pi_{n,s}(d) \tag{1}$$

Let, $I(x_{T+1}, w_{T+1}, e_{T+1}) = \Im$, then:

$$\pi_{n,s}(d) = \begin{cases} \sum_{t=1}^{T} \left(e_t\, w_t - c_t\right) + \Im, & \text{if } powerstation \\ \sum_{t=1}^{T} \left(-c_t\right) + \Im, & \text{if } reservoir \end{cases} \tag{2}$$

Subject to:

$$w_t = \kappa(\eta\rho g(h(x_t) - h_{ps} - \gamma d_{t,n}^2) d_{t,n}) \tag{3}$$

$$d_{min,n} \leq d_t \leq d_{max,n} \tag{4}$$

$$x_{n,min} \leq x_{n,t} \leq x_{n,max} \tag{5}$$

$$x_{n,t} = x_{n,t-1} - \delta d_{t,n} + \delta \sum_{j=1}^{N_u} (o_{j,t} + q_{j,t} + d_{j,(t-\tau)}) \tag{6}$$

where:

e_t	$\longrightarrow$	Price for energy [Euro/MWh]
e_{T+1}	$\longrightarrow$	Rest price: expected price after the planning horizon [Euro/MWh]
w_t	$\longrightarrow$	Power production [MWh]
c_t	$\longrightarrow$	Costs for violating reservoir levels, or start-stop [Euro]
η	$\longrightarrow$	Turbine and generator efficiency [fraction, unit free]
ρ	$\longrightarrow$	Density of water [1000 kg/m^2]
g	$\longrightarrow$	Gravity [9.81 m/s^2]
h	$\longrightarrow$	Water elevation height at reservoir or powerstation [m]
γ	$\longrightarrow$	Friction loss coefficient [s^2/m^5]
$d_{t,n}$	$\longrightarrow$	Water release decision from powerplant or reservoir [m^3/s]
$x_{n,t}$	$\longrightarrow$	Filling in reservoir n, [m^3]
I	$\longrightarrow$	Expected profit of residual water [Euro]
q	$\longrightarrow$	Natural inflow to node [m^3/s]
o	$\longrightarrow$	Overflow from upstream node (reservoir) [m^3/s]
T	$\longrightarrow$	Number of time stages
N	$\longrightarrow$	Number of nodes, (N_u number of upstream nodes)
κ	$\longrightarrow$	Daily time step conversion factor (24*3600)/(1000000*3600)
δ	$\longrightarrow$	Time step length (24*3600) [s]

V. METHODS

The general idea in this work is to apply an optimization-simulation based approach to resolve the problem specified in section 4 (see Fig. 2). We propose to use a set of standard meta-heuristic optimization/search methods like Hill Climbing (HC) [18], Simulated Annealing (SA) [11] and Line Search (LS) [8] as well as evolutionary algorithms like the Genetic Algorithm (GA) [9]. We also try out a combination of the two called memetic algorithms [16]. Finally we propose a novel meta-heuristic method for fast optimization. We call it Split-window method. It is a simple and effective non-gradient optimization method that provides a good solution with low complexity.

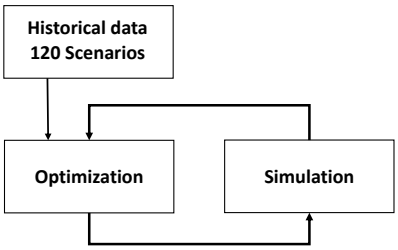


Fig. 2. Illustration of the general optimization-simulation methodology.

In total we try out seven different algorithms that tries to identify the optimal release decisions for a given scenario. They all use the principal of investigating the search-space by setting release decisions (actions), simulating the consequence of them, and getting the total profit as the feedback from the simulated environment. In practice this means finding the hatch release of water from the upper reservoir, and the production level at the Kvinesdal powerstation. More details of the various methods are specified below.

A. Rank Price Initialization (RPI)

For several of the algorithms used in this research, there is a need for initialization of the starting actions. Two different methods, Rank Price Initialization (RPI), and Random Initialization (RI) was used for this purpose. The RPI method works in the following way: At first all actions were set to zero and total profit was calculated. Then all actions were set to full capacity (1.0) and profit was calculated again. After this the rank of the price was used to initialize the actions to the same value in all nodes (hatch and production levels) according to Eq. 7.

$$a_n = R^x, \qquad x = [0, 1, 2...20] \tag{7}$$

When x increases more production will be put on the days with highest price, and less on days with lower prices. When x is equal to twenty, 90 % of the days will have actions below 0.13. For each of the x values the actions were set and profit was calculated. The actions yielding highest profit was kept as the best solution throughout all simulations. This method will be named RPI throughout the rest of this paper.

The RI, simply apply hundred random actions , try them out and identifies the one with best performance. The reason for choosing hundred was to keep computational demand at a minimum level.

In both the RPI and RI methods the sequence of actions giving highest total profit was kept as the best solution. Both the HC, LS and SA was initialized with the RPI and RI method before the main search started. This was to ensure that a reasonable starting point was chosen.

B. Meta-heuristic Methods

The Simulated Annealing algorithm is shown in Algorithm 1. P_1 (hatch release) and P_2 (production level) are initialized with the RPI and RI methods. Random values between -1 and 1 are added to both P_1 and P_2 for each day in the scenario and the objective function output is calculated using eq 1.

SIM_ANNEAL($Data$):
Initialize: All $P_1, P_2 \qquad withRPI, RI$
s =*number of steps in planning horizon*
$F_{best} = 0.0$
while $j \neq$ *EPOCHS* **do**
  for *i,* $i < s$, *i+=1* **do**
    $T = EPOCHS/i$
    $P_1[i] += random(-1, 1)$
    $P_2[i] += random(-1, 1)$
    Keep P_1 and P_2 between 0 and 1.
    Calculate objective function output F using eq 1.
    if $F \leq F_{best}$ **then**
      if $e^{-(F_{best}-F)/T} < random(0, 1)$ **then**
        continue
      end
    end
    $F_{best} \longleftarrow F$
  end
  Update $Data$ with P_1 and P_2.
end

Algorithm 1: Simulated Annealing

Unlike hill climbing, where updates are accepted only if they improve the solution (profit), while in simulated annealing, updates that result in a performance decline are sometimes accepted according to the temperature parameter T in order to avoid getting stuck in a local optimum.

In this work we also apply the line search method, which is a modified gradient based method that solves the problem of divergence in gradient ascent. It has the same update rule as gradient ascent:

$$x \longleftarrow x + \eta \nabla f(x) \tag{8}$$

Unlike in gradient ascent, η sized steps are taken in the rising gradient's direction, until f starts to decrease. This is then the new current state.

C. Genetic Algorithm

The genetic algorithm is a part of a class of algorithms called evolutionary algorithms which were inspired by Charles Darwin's theory of natural selection and evolution [9].
We design a simple genetic algorithm, tailored to resolve the hydropower optimization problem at hand. The stages of the genetic algorithm are explained below. Note that all the operations are on P_1 and P_2.

- *Population Generation:* The population consisting of N number of datasets are generated with random values for P_1 and P_2. We also added one member with all actions turned off (0.0), and one member with all actions turned on (1.0). Additionally, twenty-one members were added using the RPI method described earlier. In the first iteration we started with 100 members.
- *Fitness function:* Here, the fitness function in the GA method is the profit value obtained from the objective function given in eq 1.
- *Selection*: The criteria for selection is simple: select the datasets from the population with the top n max fitness function values. In this work, the upper half of the members were selected.
- *Crossover*: We use two-point crossover, where the second point is the last instance of the dataset (parent). A random position in the parents is chosen and the all the values after that swapped between the parents. Ten new members are added to the population every epoch.
- *Mutation*: A dataset is mutated at random out of the existing members of the population, by adding random noise between -0.05 and 0.05 to all P_1 and P_2 values. For each main iteration 10 new mutated members were added to the population.

D. Memetic Algorithms

Memetic algorithms are a set of hybrid methods that incorporate population based global search techniques like evolutionary algorithms in combination with local refinement based meta-heuristic methods [16]. We propose to add a stage with hillclimbing and/or simulated annealing to the genetic algorithm. Since the genetic algorithm described in the previous section lacks an update step (crossover is just a random swapping of datasets and mutation occurs regardless of whether the change is in the right direction of the optimum or not), we propose three ways to incorporate use of HC or SA as an update step.

1) Genetic Algorithm with Hill Climber as update step (GA+HC): In this method, the genetic algorithm is run as explained in the previous section. After the step with mutation an extra updating round of hillclimbing is carried out on each members actions. This will most often improve the performance of all members in the population.

2) Genetic Algorithm with Simulated Annealing as an additional stage (GA+SA): Here, we incorporate simulated annealing inside the genetic algorithm after an update step with the Hillclimber. For each member of the population we perform one epoch of Simulated Annealing.

As simulated annealing provides genetic algorithm with a much needed update step, it can be argued that the mutation phase can be removed from the algorithm. One can also argue that, the crossover phase can also be removed by the same logic. However, in our experiments we found that removing the crossover phase adversely affects performance. We conjecture that crossover is needed for exploration and prevents the algorithm from stagnating to a local optimum.

3) Genetic Algorithm with Simulated Annealing stage only on the best dataset (GAAB): Since, running simulated annealing on every dataset received from the previous stage notably increases execution time, we restrict simulated annealing to optimize only the best dataset (maximum fitness value) from the previous stage. This remarkably reduces execution time and space complexity.
The initialization method used for all the population generation in the memetic algorithms is the same as for the genetic algorithm.

E. Split-window Algorithm

In this paper, we propose a novel meta-heuristic method, called the split-window algorithm. It was designed with the intention of having low computational demands while not compromising performance. The pseudo-code for the split-window method is shown in Algorithm 2.

The planning horizon is split into 2^n equal parts/windows in n iterations. For each window, all instances (all days in each part) have the same P_1 values and P_2 values. In the first iteration there are two windows, with two free actions (hatch release and production level) in each window, providing a total of four free parameters to optimize. The optimization is then carried out with the HC algorithm optimizing only these four actions, which are set for the every step in the window. In the second iteration there are four windows, and eight free actions, and so on.

In the last stage of iterations the number of windows are the same as the number of time steps in the planning horizon. The algorithm then behaves like a normal hill climber, but receives near-optimal initialized actions. This makes the split-window technique relatively fast and efficient since it does not require an explicit initialization overhead.

The split-window algorithm is quite flexible as any meta-heuristic method can be used with it. The first $n-1$ iterations can be thought of as initialization steps for the nth iteration, which is the meta-heuristic method.

SPLIT-WINDOW($Data$):
Initialize: All $P_1, P_2 = 0$
s =*number of days*
$F_{best} = 0.0$
$j = 2$
while $j \leq s$ **do**
  $k = Quotient(s, j)$
  for q, $q \leq s + 1$, q+=k **do**
    if q==*0* **then**
      continue
    end
    while *epoch* $\neq$ *EPOCHS* **do**
      $\delta_1 = random(-1, 1)$
      $\delta_2 = random(-1, 1)$
      for *i*, $i < q$, *i*+=*1* **do**
        $P_1[i] += \delta_1$
        $P_2[i] += \delta_2$
        Keep P_1 and P_2 between 0 and 1.
      end
      Calculate objective function output F using eq 1.
      if $F \leq F_{best}$ **then**
        continue
      end
      $F_{best} \longleftarrow F$
    end
    Update $Data$ with P_1 and P_2.
  end
  $j \longleftarrow j^2$
end

Algorithm 2: Split-window

VI. Experiments and Results

We exhaustively test all the algorithms mentioned in the previous section on 120 different scenarios extracted from the real world hydrologic and energy price data. We use a 180 days future window as the planning horizon. The scenarios were selected using the first day in the month as the starting point. The time period from February 1, 2007 until January 1, 2017 gave a total of 120 historical scenarios. By selecting scenarios in this way we ensured that the all the algorithms were tested for a large range of different inflow and price situations. The starting reservoir level and rest-price were also taken from the historical data.

We run simulations for all the algorithms until they converge to their maximum profit value, i.e., until there is no or very little change in profit. For each of the algorithms used we counted the number of times profit was simulated together with the development in profit. Each of the 120 scenarios would then get a record with profit development. From all of these 120 scenarios average profit development was calculated.

To minimize effects of random initialization all algorithms were run from start to finish one hundred times (not the same as RI method described earlier). Based on this we could find the maximum profit development as well as the minimum or "worst" profit development.

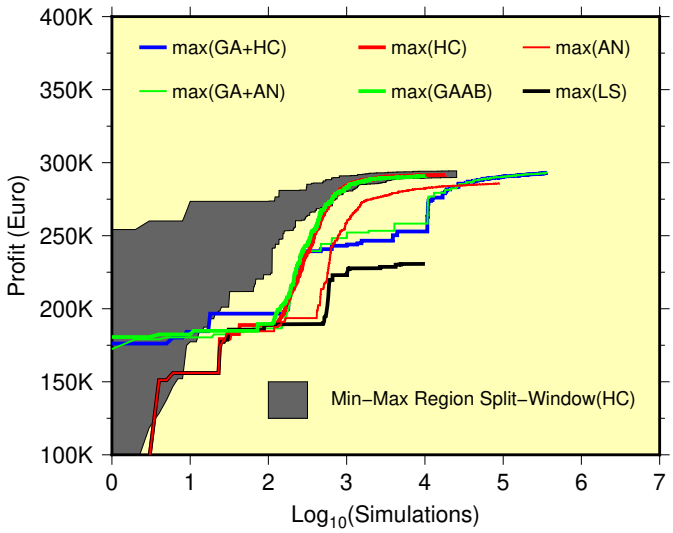


Fig. 3. Average region of performance of Split-window vs. Other Meta-heuristic and Memetic algorithms

Figure 3 shows a graphical representation of the results for: Split-window with HC, Hill Climbing (HC), Simulated Annealing (AN), Line Search (LS), Genetic Algorithm with Hill Climbing (GA+HC), Genetic Algorithm with Simulated Annealing (GA+AN) and Genetic Algorithm with Simulated Annealing only on the Best dataset (GAAB). All the results shown in the graph represent max performances of all algorithms. However, for the case of Split-window, we plot the region bounded by minimum and maximum performance.
From the graph in Figure 3, we can see that even in the worst case the split-window algorithm is able to reach a higher profit than all the other algorithms in less than 100 ($log_{10}2$) simulations. The amount by which the split-window outperforms other methods is more profound in terms of how quickly it can achieve higher profit scores. From there on out, it performs better than the rest of the methods. So, in order to reach a high enough profit, the split-window is the preferred option since it is able to achieve a higher profit value in the least number of simulations. In addition, the proposed split-window technique achieves the highest total profit.
It is difficult to compare these algorithms in terms of total execution time and full computational complexity since each method has some overhead attached to how it selects actions and way of initialization that suits a particular algorithm. However, the proposed split-window method is able to beat other techniques in terms of performance score, speed of convergence and complexity per iteration by a considerable margin.

VII. Conclusion

In this paper we have proposed as novel technique, called split-window to optimize hydropower generation. The split-

window method is computationally less expensive than other optimization techniques and can be thought of as an initialization method for meta-heuristic algorithms. We also propose a few memetic algorithms that are a combination of genetic algorithms and meta-heuristic methods. As shown from the results, memetic algorithms perform better than genetic algorithms as well as meta-heuristics. Furthermore, our proposed split-window algorithm outperforms all evaluated methods in terms of profit score as well as speed. It is able to reach high amounts of profit much more quickly compared to any of the other techniques. Even the worst case performance of the split-window is comparable to the other optimization methods.

VIII. Acknowledgment

This research was supported by Agder Energi AS, the Norwegian Research Council (EnergiX) and the University of Agder. We also acknowledge feedback on this work from E. S. Huse at Agder Energi.

References

[1] Ahmad A., El-Shafie A., Razali S.F.M., and Mohamad Z. S. Reservoir optimization in water resources: a review. *Water Resour Manage*, 27:3391–3405, 28 2014.

[2] Castelletti A., Galelli S., Restelli M., and Soncini-Sessa R. Tree-based reinforcement learning for optimal water reservoir operation. *WATER RESOURCES RESEARCH*, 46, 2010.

[3] Allen B. and Bridgeman R. G. Dynamic programming in hydropower scheduling. *Journal of Water Resources Planning and Management*, 1986.

[4] Matheussen B.V and Sharma J. Granmo O-C. Hydropower optimization using deep learning. *Lecture notes in Artificial Intelligence 32nd International Conference on Industrial, Engineering Other Applications of Applied Intelligent Systems- In press*, 2019.

[5] DeRigo D., Rizzoli A.E., Soncini-Sessa R., Weber E., and Zenesi P. Neuro-dynamic programming for the efficient management of reservoir networks. *Proceedings of MODSIM 2001 International Congress on Modelling and Simulation*, 4:1949–1954, December 2001.

[6] Moradi A. M. Dariane A. B. Comparative analysis of evolving artificial neural network and reinforcement learning in stochastic optimization of multi-reservoir systems. *Hydrological Sciences Journal*, 61:1141–1156, 2016.

[7] Sabah Fayaed, Ahmed El-Shafie, and Othman Jaafar. Reservoir-system simulation and optimization techniques. *Stochastic Environmental Research and Risk Assessment*, 27, 10 2013.

[8] Bozorg-Haddad O. Garousi-Nejad, I. and H. Loaiciga. Modified firefly algorithm for solving multi-reservoir operation in continuous and discrete domains. *Journal of Water Resources Planning and Management*, 142, September 2016.

[9] David E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1st edition, 1989.

[10] Lee J.H. and Labadie J.W. Stochastic optimization of multi-reservoir systems via reinforcement learning. *WATER RESOURCES RESEARCH*, 43, 2007.

[11] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[12] Azizipour M., Ghalenoei V., Afshar M. H., and Solis S. S. Optimal operation of hydropower reservoir systems using weed optimization algorithm. *Water Resour Manage*, 30:3995–4009, 2016.

[13] Sangiorgio M. and Guariso G. Nn-based implicit stochastic optimization of multi-reservoir systems management. *Water*, March 2018.

[14] Afshar M.H. and Motaei I. Constrained big bang big crunch algorithm for optimal solution of large scale reservoir operation problem. *Int. J. Optim. Civil Eng.*, 2:357–375, 2011.

[15] Belsnes M.M., Wolfgang O., Follestad T., and Aasgard E.K. Applying successive linear programming for stochastic short-term hydropower optimization. *Electric Power Systems Research*, 130:167–180, January 2016.

[16] Pablo Moscato. New ideas in optimization. chapter Memetic Algorithms: A Short Introduction, pages 219–234. McGraw-Hill Ltd., UK, Maidenhead, UK, England, 1999.

[17] Fosso O.B and Mo B. Wangensteen I. Gjelsvik A., Haugstad A. Generation scheduling in a deregulated system. the Norwegian case - discussion. *IEEE Transactions on Power Systems*, 14(1):75–82, 1999.

[18] Norvig P. Russel S. *Artificial Intelligence A modern Approach, Third Edition*. Prentice Hall, 2010.

[19] Moosavian S.A.A. and Salimi A. Ghaffari A. Sequential quadratic programming and analytic hierarchy process for nonlinear multiobjective optimization of a hydropower network. *Optimal Control Applications and Methods*, 2010.

[20] Zhao T., Zhao J., and Yang D. Improved dynamic programming for hydropower reservoir operation. *Journal of Water Resources Planning and Management*, 140, 2014.

[21] Archibald T.W. and Marshall S.E. Review of mathematical programming applications in water resource management under uncertainty. *Environmental Modelling Assessment*, pages 1–1, 2018.

[22] Jothiprakash V. and Arunkumar R. Multi-reservoir optimization for hydropower production using nlp technique. *Journal of Civil Engineering*, 18, January 2014.

[23] Labadie J. W. and Greenwade G. D. Optimal operation of multi-reservoir systems state-of-the-art review. *Journal of Water Res. Planning and Management*, 130(2):93–94, 2004.

[24] Ju-Hwan Yoo. Maximization of hydropower generation through the application of a linear programming model. *Journal of Hydrology*, 376:182–187, 2009.